

Reviewer Pack — Algebraic K-Theory Rank and SAT Clause Set Complexity Correl...

Entry 84c5cca89575 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 04:03:22 UTC

Algebraic K-Theory Rank and SAT Clause Set Complexity Correlation

Entry ID: 84c5cca89575 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 04:03:22 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-Theory **Field B** (complexity object): Boolean Satisfiability (SAT)

Statement:

For every Boolean satisfiability instance with n variables, the algebraic K-theory rank of the corresponding ring $R(\text{SAT})$ generated by the clause set is linearly correlated with the clause set complexity $c(\text{SAT})$, such that $|K_0(R(\text{SAT}))| = \Theta(c(\text{SAT}))$.

Rationale (proposer's reasoning):

Algebraic K-theory provides a framework for studying the structure of rings, and its rank could offer insights into the complexity of SAT instances. A direct correlation between these invariants might expose underlying structural properties of SAT that are not easily accessible through traditional complexity measures.

Taxonomy category: K0-algebraic-geometry (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 25880255d863452b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $|K_0(R(\text{SAT}))|$ and $c(\text{SAT})$ exceeds 0.7 for all seeds, with no seed yielding a correlation coefficient below -0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.90	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'Algebraic K-Theory rank' AND 'Boolean Satisfiability (SAT)' - 'Clause set complexity' IN ('algebraic K-theory rank' OR 'K0(R(SAT))') - algebraic K-theory AND Boolean satisfiability correlation with clause set complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_random_cnf(n, m):
    cnf = []
    for _ in range(m):
        clause = [random.randint(1, n) * (-1 if random.choice([True, False]) else 1)
                   for _ in range(random.randint(1, n))]
        cnf.append(clause)
    return cnf

def compute_k0_rank(cnf):
    n = max(abs(lit) for lit in sum(cnf, []))
    rank = sum(1 for clause in cnf if any(lit in clause for lit in [-i, i] for i in range(1, n+1)))
    return rank

def compute_correlation_coefficient(x, y):
    n = len(x)
    mean_x = sum(x) / n
    mean_y = sum(y) / n
    numerator = sum((x[i] - mean_x) * (y[i] - mean_y) for i in range(n))
    denominator = math.sqrt(sum((x[i] - mean_x)**2 for i in range(n))) * math.sqrt(sum((y[i] - mean_y)**2 for i in range(n)))
    return numerator / denominator if denominator != 0 else 0

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    ranks = []
    complexities = []

    for n in n_values:
        cnf = generate_random_cnf(n, n * (n - 1) // 2)
        rank = compute_k0_rank(cnf)
```

```

complexity = len(cnf)

if rank is None or complexity is None:
    return {
        "metric_name": "K0 Rank vs Clause Set Complexity",
        "metric_value": None,
        "instances_tested": len(n_values),
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

ranks.append(rank)
complexities.append(complexity)

correlation_coefficient = compute_correlation_coefficient(ranks, complexities)
conjecture_holds = -0.7 < correlation_coefficient < 0.7

return {
    "metric_name": "K0 Rank vs Clause Set Complexity",
    "metric_value": correlation_coefficient,
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"Correlation coefficient: {correlation_coef
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2*i + 3 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\\"seed\\": {seed}, ...{result}...}}")
        results.append(result)

    if all("conjecture_holds": True for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
    elif any(result["conjecture_holds"] is False for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
        print(f"RESULT: FALSIFIED counterexample=\\\"{result['counterexample']}\\\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing *TRIAL:* lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0391b3b4.py", line 92, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0391b3b4.py", line 42, in run_trial
    rank = compute_k0_rank(cnf)
            ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0391b3b4.py", line 30, in compute_k0_rank
    rank = sum(1 for clause in cnf if any(lit in clause for lit in [-i, i] for i in range(1, n+1)))
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0391b3b4.py", line 30, in <genexpr>
    rank = sum(1 for clause in cnf if any(lit in clause for lit in [-i, i] for i in range(1, n+1)))
                                                    ^
NameError: name 'i' is not defined. Did you mean: 'id'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined variable 'i', which prevented the computation of the algebraic K-theory rank and correlation coefficients. | next: Review the code for the error and correct the definition of 'i' or any other necessary corrections before re-running the test.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16685
2	preregistration	ollama_remote	glm4:latest	0	0	9286
3	novelty	ollama_remote	glm4:latest	0	0	8885
4	novelty	ollama_remote	glm4:latest	0	0	19896
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16008
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10211
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29770
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23643
9	judge	ollama_remote	glm4:latest	0	0	11676

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 146060 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/84c5cca89575.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/84c5cca89575.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/84c5cca89575.tar.gz` (if generated)